Polytech

Optimisation Discrète

Projet

Problème du sac à dos

Présentée par

Julien Larzul & Victor Lefevre

Le 16 mai 2025

Polytech Lyon
15 Bd André Latarjet,
69100 Villeurbanne

Professeur,
Stéphane Bonnevay

Table des matières

Introduction **3**
Interface de jeu **4**
Présentation des algorithmes **5**
Expérimentations et résultats **7**
Conclusion **14**

Introduction

Dans le cadre du projet d'intelligence artificielle de la filière Informatique, l'objectif est de concevoir une intelligence artificielle capable de jouer à un jeu à deux joueurs à information complète, en mettant en œuvre plusieurs algorithmes de recherche et de décision. Parmi les algorithmes étudiés dans ce projet, nous nous concentrons sur Minimax, Alpha-Beta et MCTS, trois approches fondamentales dans la prise de décision pour des jeux.

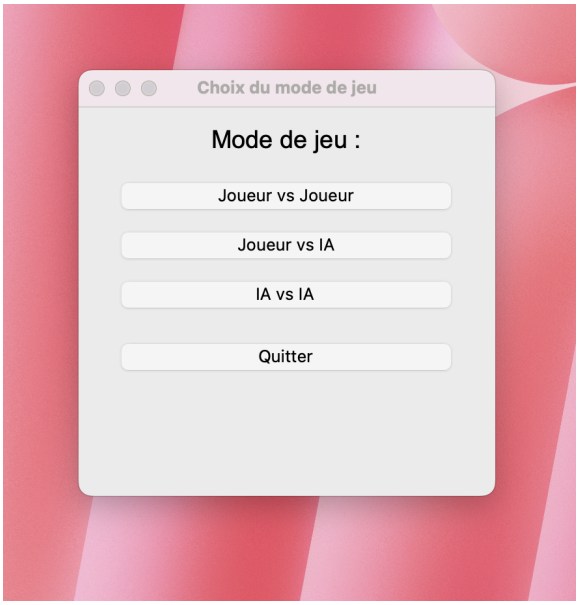
Afin de répondre aux objectifs du projet, le jeu Puissance 4 a été choisi comme environnement d'expérimentation. Ce jeu, bien connu et facile à modéliser, présente l'avantage de disposer d'un espace d'états suffisamment complexe pour illustrer les différences d'efficacité entre les algorithmes tout en restant accessible à l'analyse.

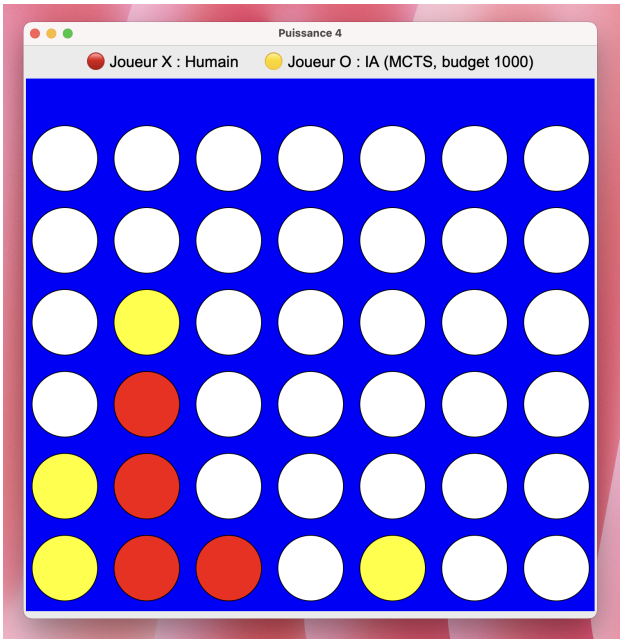
Ce rapport présentera dans un premier temps la modélisation du jeu et les fonctions d'évaluation mises en place. Nous détaillerons ensuite l'implémentation des algorithmes Minimax, Alpha-Beta et MCTS, en expliquant leur fonctionnement, leurs performances respectives, ainsi que les optimisations possibles. Enfin, des tests comparatifs seront menés afin d'évaluer ces trois algorithmes selon différents critères : profondeur de recherche, temps de calcul, nombre de nœuds explorés et qualité des décisions prises.

Interface de jeu

Dans un premier temps, nous avons développé une interface console permettant de tester rapidement les différents algorithmes d'IA. Ce mode, simple à utiliser, a facilité le débogage, la validation des algorithmes et l'automatisation des tests IA contre IA.

Dans un second temps, nous avons réalisé une interface graphique (GUI) en Python avec Tkinter. Cette interface permet à l'utilisateur de jouer contre l'IA ou de faire jouer deux IA l'une contre l'autre, tout en choisissant l'algorithme utilisé pour chaque joueur (Minimax, Alpha-Beta ou MCTS) ainsi que la "force" (profondeur ou budget de simulations). L'affichage en couleurs des jetons et l'ergonomie permettent de visualiser facilement le déroulement de la partie, les coups joués et l'issue du jeu.





Présentation des algorithmes

1. Algorithme Minimax

L'algorithme Minimax est un algorithme de recherche de décision qui s'applique aux jeux adversariaux. Il repose sur un principe simple mais puissant : chaque joueur tente de maximiser ses gains tout en supposant que son adversaire agit de manière parfaitement rationnelle pour minimiser ces mêmes gains.

Concrètement, l'algorithme explore récursivement toutes les configurations possibles du jeu jusqu'à une certaine profondeur. À chaque nœud de l'arbre de recherche, il applique alternativement une stratégie de maximisation ou de minimisation selon le joueur courant. Lorsqu'il atteint une feuille de l'arbre (c'est-à-dire un état final ou une profondeur limite), il évalue la situation à l'aide d'une fonction d'évaluation heuristique.

Minimax est efficace pour identifier la meilleure séquence de coups, mais il souffre d'un coût computationnel élevé. En effet, le nombre d'états à analyser augmente exponentiellement avec la profondeur d'exploration. Cela limite son applicabilité à des profondeurs modestes dans des jeux comme Puissance 4, où chaque coup offre plusieurs possibilités.

2. Algorithme Alpha-Bêta

L'algorithme Alpha-Bêta est une amélioration directe de Minimax. Son objectif est de réduire le nombre de nœuds explorés dans l'arbre sans compromettre la qualité de la décision prise. Pour cela, il introduit deux bornes de coupure : alpha pour le joueur maximisant et bêta pour le joueur minimisant.

Lors de l'exploration de l'arbre, si une branche ne peut pas améliorer le résultat optimal déjà trouvé à un niveau supérieur, elle est élaguée, c'est-à-dire ignorée. Cela permet d'éviter l'analyse de sous-arbres entiers qui n'ont aucune chance d'être choisis.

En pratique, Alpha-Bêta permet d'atteindre des profondeurs plus importantes que Minimax avec les mêmes ressources, tout en garantissant le même résultat final. L'efficacité de cette technique dépend cependant de l'ordre dans lequel les coups sont examinés. Si les meilleurs coups sont analysés en premier, les coupures sont maximisées.

3. Algorithme Monte Carlo Tree Search (MCTS)

L'algorithme Monte Carlo Tree Search adopte une approche totalement différente des deux précédents. Au lieu d'évaluer systématiquement chaque coup possible selon un schéma déterministe, MCTS repose sur la simulation aléatoire de parties complètes à partir d'un état donné.

Son fonctionnement se divise en quatre étapes principales :

1. **Sélection** : un chemin est parcouru dans l'arbre en suivant un équilibre entre exploration et exploitation.

2. **Expansion** : un nouveau nœud (état) est ajouté à l'arbre.

3. **Simulation** : une partie aléatoire est jouée à partir de ce nœud jusqu'à une fin de partie.

4. **Rétropropagation** : les résultats de cette simulation sont propagés vers le haut de l'arbre pour mettre à jour les statistiques des nœuds visités.

Au fil des itérations, l'arbre de recherche se concentre sur les coups les plus prometteurs. Contrairement à Minimax et Alpha-Bêta, MCTS ne nécessite pas de fonction d'évaluation spécifique, ce qui le rend particulièrement adapté aux jeux complexes ou mal modélisés. En revanche, son efficacité dépend fortement du nombre de simulations effectuées (le "budget").

Expérimentations et résultats

1. Comparaison Minimax vs Alpha-Beta

Pour comparer les performances des algorithmes Minimax et Alpha-Beta, nous avons réalisé une série de tests identiques pour chacun.
Les expérimentations consistent à faire s'affronter deux IA utilisant le même algorithme (Minimax vs Minimax, puis Alpha-Beta vs Alpha-Beta), tout en faisant varier la profondeur de recherche.

Pour chaque match, nous avons relevé deux indicateurs clés pour chaque joueur :

- Le temps total d'exécution (temps mis par l'IA pour jouer l'ensemble de ses coups),
- Le nombre total de nœuds explorés durant la partie.

Ces mesures permettent de comparer non seulement l'efficacité computationnelle des deux algorithmes, mais aussi leur comportement en fonction de la profondeur. En maintenant des conditions identiques entre les tests (même positions initiales, mêmes profondeurs), nous assurons une comparaison équitable entre les deux méthodes.

Voici le tableau de résultat pour minimax:

Prof X	Temps X (s)	Nœuds X	Prof O	Temps O (s)	Nœuds O	Gagnant
1	0.0203	93	2	0.1254	577	O
2	0.1464	702	2	0.1369	668	X
2	0.1518	754	5	33.2373	172313	O
3	0.8064	3844	3	0.7519	3742	O
3	0.9703	4682	5	31.9259	169762	O
4	4.8934	24826	4	4.791	24012	O
5	36.9565	193456	5	33.7741	177575	X

Et voici le tableau de résultat pour Alpha-Beta :

Prof X	Temps X (s)	Nœuds X	Prof O	Temps O (s)	Nœuds O	Gagnant
1	0.0217	93	2	0.0761	331	O
2	0.1163	533	2	0.1062	472	X
2	0.1156	558	5	10.3699	49871	O
3	0.3591	1615	3	0.3064	1369	O
3	0.4392	1919	5	4.8041	21766	O
4	1.6815	7699	4	1.6758	7707	O
5	6.3588	27400	5	6.4596	27593	X

Les deux algorithmes ont été testés dans des conditions identiques, avec des profondeurs variant de 1 à 5. Le but était de mesurer leur temps total de calcul et le nombre de nœuds explorés.

Comparaison des performances :

- À profondeur 2, les deux algorithmes produisent un comportement similaire sur le plan stratégique. Cependant, Alpha-Beta explore 472 à 533 nœuds contre 668 à 702 pour Minimax.

- À profondeur 3, Minimax explore environ 3 800 à 4 600 nœuds et met près d'une seconde pour jouer. Alpha-Beta, lui, n'explore que 1 300 à 1 900 nœuds, pour un temps de moins de 0.5 seconde. Le gain est donc aussi bien en temps qu'en espace de recherche.

- À profondeur 5, les différences deviennent énormes : Minimax peut monter jusqu'à 170 000 à 190 000 nœuds explorés, avec des temps d'exécution entre 30 et 36 secondes. Alpha-Beta, quant à lui, reste autour de 27 000 à 49 000 nœuds, et termine en 6 à 10 secondes. L'optimisation est donc clairement démontrée.

Cette efficacité d'Alpha-Beta s'explique par son principe d'élagage : il coupe des branches entières de l'arbre de recherche dès qu'il détecte qu'elles ne peuvent pas influencer la décision finale. Cela permet de réduire drastiquement le nombre de nœuds parcourus sans modifier la décision prise à l'issue de la recherche.

Sur un second plan, on peut analyser les performances des deux algorithmes. Nous observons que, pour des parties identiques jouées avec les mêmes profondeurs, Minimax et Alpha-Beta produisent systématiquement les mêmes résultats (même joueur gagnant).
Cela s'explique par le fait que les deux algorithmes utilisent exactement la même fonction d'évaluation. Cette fonction attribue un score aux alignements de 4 cases en fonction du nombre de pions du joueur et de l'absence de pions adverses. Elle valorise donc fortement les positions gagnantes ou menaçantes, tout en ignorant les lignes déjà bloquées par l'adversaire.

Cette fonction a également un impact direct sur la performance d'Alpha-Beta. En effet, plus les scores sont discriminants et rapidement exploitables, plus l'algorithme est en mesure de réaliser des coupures précoces dans l'arbre de recherche. Une fonction d'évaluation bien pensée ne se contente donc pas de guider la stratégie de jeu ; elle renforce aussi l'efficacité du parcours de l'arbre, en permettant à Alpha-Beta de réduire considérablement le nombre de nœuds explorés tout en conservant une qualité de décision équivalente à Minimax.

2. Test et analyse de MCTS

L'IA explore les coups possibles en simulant aléatoirement des parties et en construisant dynamiquement un arbre de recherche, afin de choisir le coup offrant la meilleure espérance de gain.

Notre implémentation s'appuie sur :

- Une classe d'arbre de jeu qui mémorise les nœuds et leurs statistiques (visites, scores…)
- Une phase d'exploration/sélection (UCB/UCT), d'expansion, de simulation et de rétropropagation du score.
- Un paramètre "budget" permettant de faire varier le nombre de simulations avant de prendre une décision.

Pour répondre à la consigne, nous avons fait varier le budget de simulations et relevé les propriétés de l'arbre généré par MCTS à chaque coup :

- **Nombre total de nœuds créés**
- **Profondeur maximale atteinte**
- **Répartition des nœuds par niveau**
- **Taille de la plus grande branche**
- **Temps d'exécution**

Dans cet exemple on a fait jouer un MCTS avec un budget de 1000 contre un avec un budget de 500 :

```
[IA X] réfléchit...
[DEBUG MCTS] Infos arbre: {'nb_noeuds': 1001, 'profondeur_max': 7, 'noeuds_par_niveau': {0: 1, 1
: 7, 2: 14, 3: 97, 4: 307, 5: 327, 6: 223, 7: 25}, 'taille_plus_grande_branche': 7, 'temps_execu
tion': 1.7362401485443115, 'iterations': 1000}
```

```
[IA O] réfléchit...
[DEBUG MCTS] Infos arbre: {'nb_noeuds': 501, 'profondeur_max': 7, 'noeuds_par_niveau': {0: 1, 1:
 7, 2: 21, 3: 76, 4: 165, 5: 171, 6: 48, 7: 12}, 'taille_plus_grande_branche': 7, 'temps_executi
on': 0.8957817554473877, 'iterations': 500}
```

```
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . O .
. X . . . X .
0 1 2 3 4 5 6
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . . .
. . . . . O .
. X . . . X .
0 1 2 3 4 5 6
```

On observe que lorsque le budget augmente l'arbre est plus dense : plus de nœuds, profondeur parfois légèrement plus grande. Le nombre de nœuds par niveau montre la "largeur" explorée : les premiers

niveaux (racine, premiers coups) sont plus remplis que les niveaux profonds. La taille de la plus grande branche correspond à la plus longue séquence de coups simulée sans interruption (une “partie virtuelle” complète). Le temps de calcul croît avec le budget, en restant raisonnable (quelques secondes pour 1000 simulations).

Budget (simulations)	**Nombre de Noeuds**	**Profondeur Max**	**Plus grande Branche**	**Temps en secondes**
500	501	6	6	0.883
1000	1001	8	8	1.885
1500	1501	9	9	2.759
2000	1996	10	10	3.272
5000	4984	10	10	8.772

Influence du budget de simulations MCTS sur la taille de l’arbre de recherche *(premier coup d’IA sur une grille vide ou après un seul coup joué)*

On pourrait aussi faire la même analyse sur des coups ultérieurs : la tendance serait similaire, mais avec généralement un arbre plus profond et moins large, car il y a moins de coups possibles.

L’augmentation du budget de simulations permet à l’IA d’explorer plus de possibilités et d’affiner ses choix, ce qui peut se traduire par des coups plus logiques ou moins faciles à contrer pour l’adversaire. Cependant, au-delà d’un certain seuil, le gain de performance devient négligeable au regard du temps de calcul supplémentaire.

L’algorithme MCTS repose sur la réalisation de nombreuses simulations aléatoires pour estimer la valeur des coups possibles. Cette part de hasard induit naturellement une variabilité dans le choix des coups, particulièrement lorsque le budget de simulations est faible.

- **Pour un budget faible** (par exemple 100 à 500 simulations), le résultat dépend fortement des premiers tirages aléatoires. Il est donc fréquent que l'IA ne choisisse pas systématiquement le même coup à partir d'une même situation de départ : le coup joué et la forme de l'arbre de recherche peuvent varier.
- **Pour un budget élevé** (par exemple 1000 à 5000 simulations ou plus), le "bruit" lié à l'aléatoire s'atténue : l'exploration devient plus exhaustive et les statistiques des coups se stabilisent. L'IA rejoue alors le même coup presque à chaque fois depuis la même configuration, et la structure de l'arbre généré est très similaire d'un essai à l'autre.

3. Comparaison finale

Au terme de nos expérimentations, il est possible d'établir une comparaison synthétique entre les trois algorithmes implémentés : Minimax, Alpha-Beta, et MCTS. Chacun présente des avantages et des limites spécifiques selon le contexte d'utilisation et les objectifs visés (rapidité, qualité de jeu, flexibilité…).

a) Qualité de jeu

Sur le plan stratégique, Minimax et Alpha-Beta sont équivalents lorsqu'ils sont exécutés avec la même profondeur, car ils s'appuient sur la même fonction d'évaluation. Ils produisent les mêmes coups dans les mêmes configurations, ce qui signifie que la qualité de leurs décisions est identique.

En revanche, MCTS adopte une approche probabiliste : il simule aléatoirement des parties pour estimer la qualité des coups. Cette méthode permet parfois de trouver des coups plus subtils que ceux envisagés par l'évaluation statique de Minimax/Alpha-Beta. Cependant, à faible budget de simulations, ses performances peuvent être aléatoires. À budget élevé, MCTS devient plus stable et compétitif, notamment dans des situations complexes ou tactiques.

b) Efficacité

En termes de performances, les écarts sont significatifs :

- Minimax est l'algorithme le plus simple, mais aussi le plus coûteux en calcul. Le nombre de nœuds explorés croît exponentiellement avec la profondeur.

- Alpha-Beta, grâce à son mécanisme d'élagage, explore 3 à 6 fois moins de nœuds que Minimax pour une même profondeur, tout en produisant les mêmes résultats. Cela se traduit par un temps d'exécution beaucoup plus court, ce qui permet de jouer à des profondeurs plus élevées.

- MCTS, quant à lui, permet de fixer un budget. Le temps de calcul est donc prévisible et contrôlable. Toutefois, son comportement est influencé par le facteur aléatoire et nécessite un grand nombre de simulations pour obtenir des résultats fiables.

c) Flexibilité et adaptabilité

- Minimax et Alpha-Beta dépendent entièrement de la qualité de la fonction d'évaluation, qui doit être soigneusement conçue pour refléter les bonnes priorités stratégiques.

- MCTS, au contraire, n'a pas besoin de fonction d'évaluation. Il apprend par l'expérience en simulant. Cela le rend très adaptable, notamment dans des jeux plus complexes.

Conclusion

Ce projet a permis de mettre en œuvre et de comparer plusieurs approches classiques de l'intelligence artificielle appliquées au jeu du Puissance 4, en particulier les algorithmes Minimax, Alpha-Bêta et MCTS. Nous avons constaté que Minimax, bien qu'efficace dans ses décisions, reste limité par la profondeur d'exploration, ce qui réduit ses performances sur des parties avancées. L'algorithme Alpha-Bêta surmonte en partie cette difficulté en élaguant l'arbre de recherche, ce qui permet d'atteindre des profondeurs plus importantes tout en conservant la qualité des choix.

De son côté, MCTS adopte une approche très différente, basée sur la simulation aléatoire de parties et la construction dynamique de l'arbre de recherche. Cette méthode s'est révélée particulièrement intéressante car elle ne nécessite pas d'heuristique experte et son efficacité dépend principalement du budget de simulations choisi. Nous avons observé que plus ce budget est élevé, plus les résultats de l'IA sont stables et pertinents, au prix d'un temps de calcul plus important.

En définitive, chaque approche possède ses atouts : Alpha-Bêta reste particulièrement performant sur Puissance 4 avec une bonne fonction d'évaluation, tandis que MCTS se distingue dans des contextes où il est difficile de concevoir une heuristique adaptée. Pour aller plus loin, il serait pertinent d'optimiser les fonctions d'évaluation utilisées ou d'explorer des variantes avancées de MCTS pour améliorer encore la qualité du jeu proposé par l'IA.